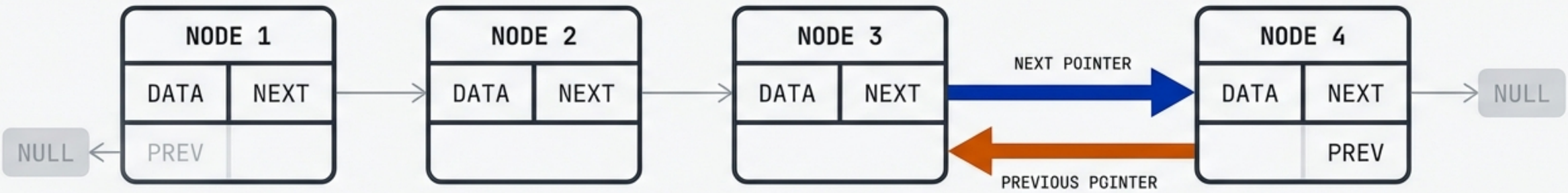
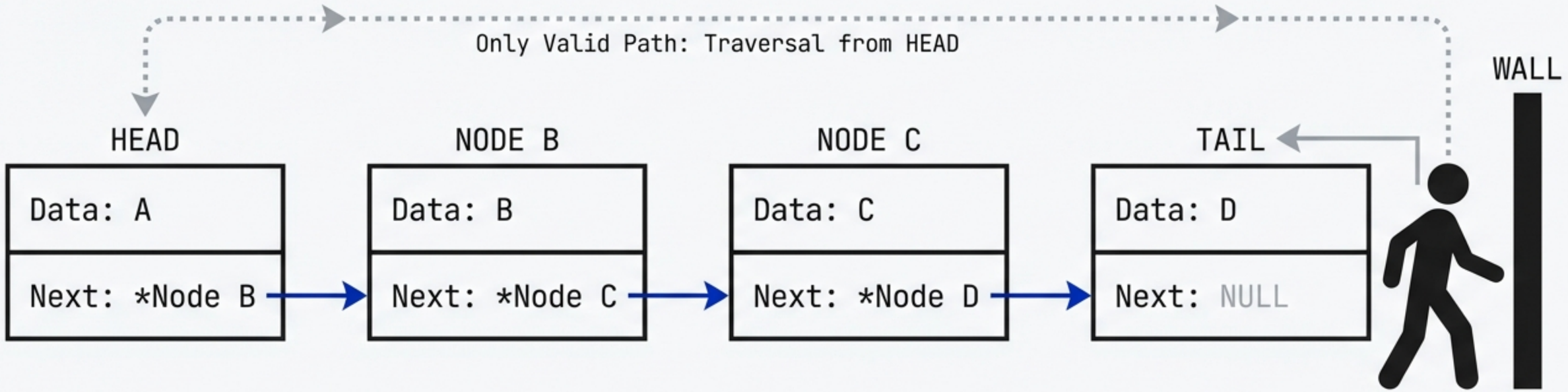
MASTERING LINKED LISTS: DOUBLY & CIRCULAR

Logic, Traversal, and The Art of Pointer Manipulation



Based on the iCodeGuru DSA Philosophy

The Limitation of the Single Path

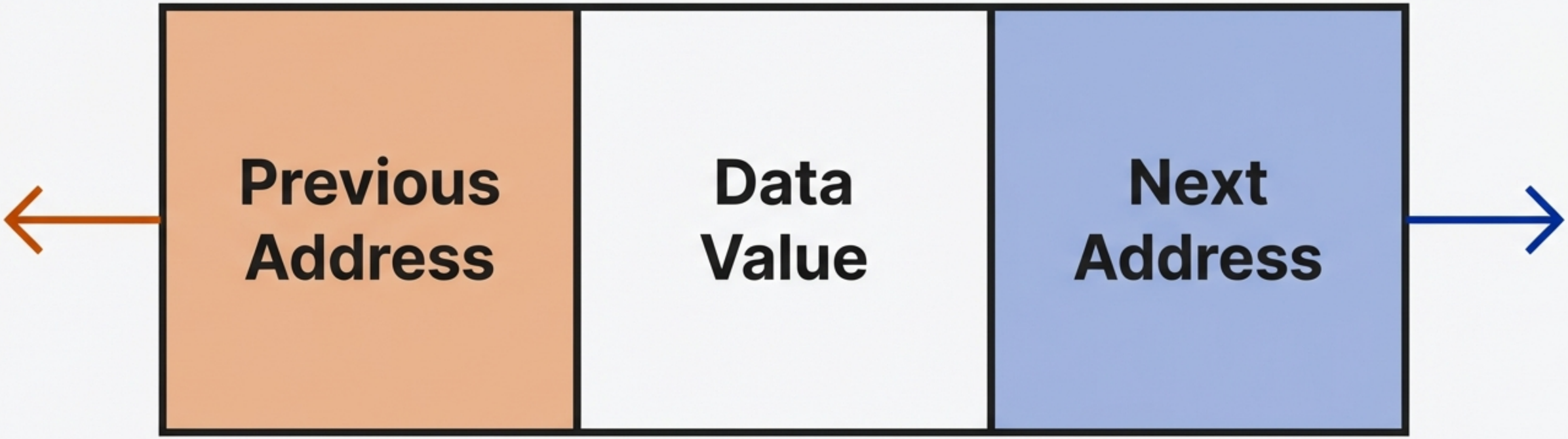


“To access the second last node... I have to traverse through every node.”

In a Singly Linked List, looking backward is impossible. Operations at the tail tail require **O(n)** traversal from the start.

Anatomy of a Bidirectional Node

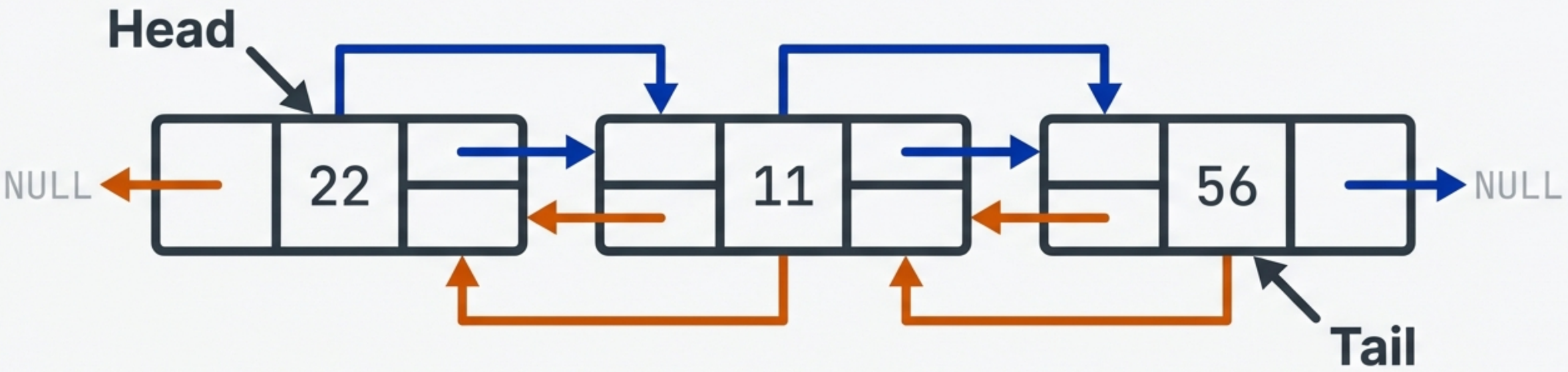
We sacrifice a small amount of memory (storing an extra address) to gain the power of bidirectional movement.




```
Node = [ Prev | Data | Next ]
```

The Doubly Linked Memory Model

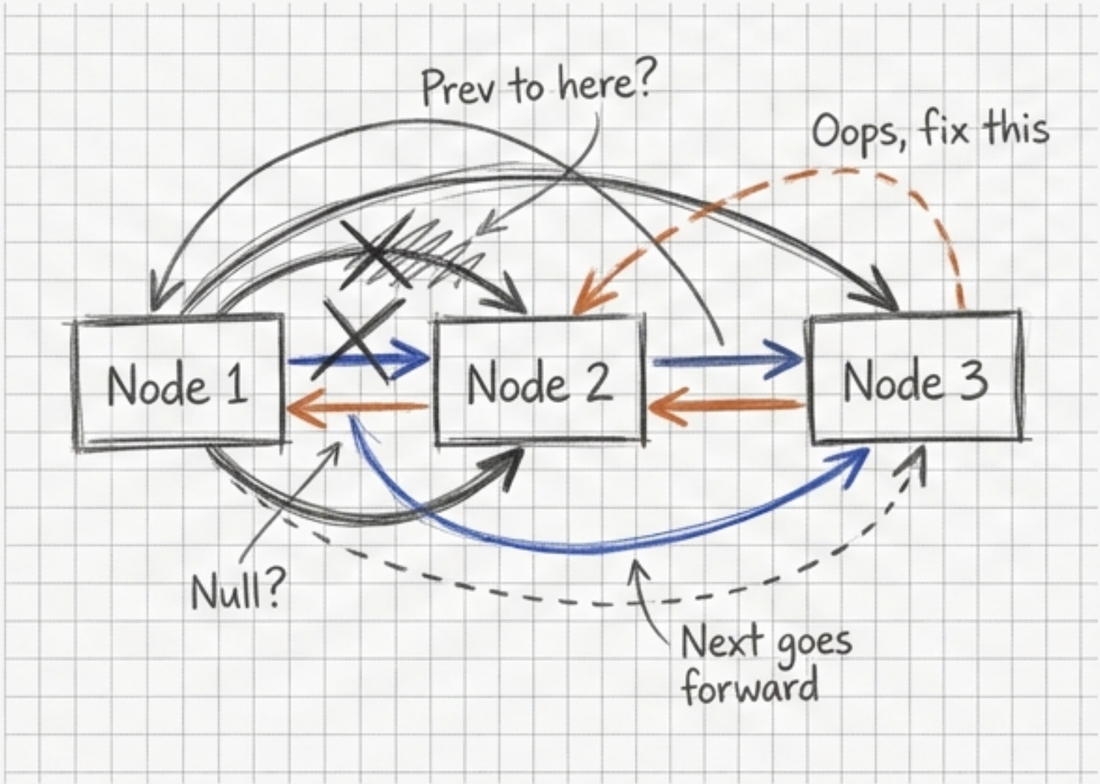
Forward or Backward. We can now execute `Tail.Previous` to immediately access the end of the list without full traversal.



Don't Code. Write the Story.

First draw it on paper. If you can write the “story” of where the arrows go, the code writes itself.

The Story (Drafting)



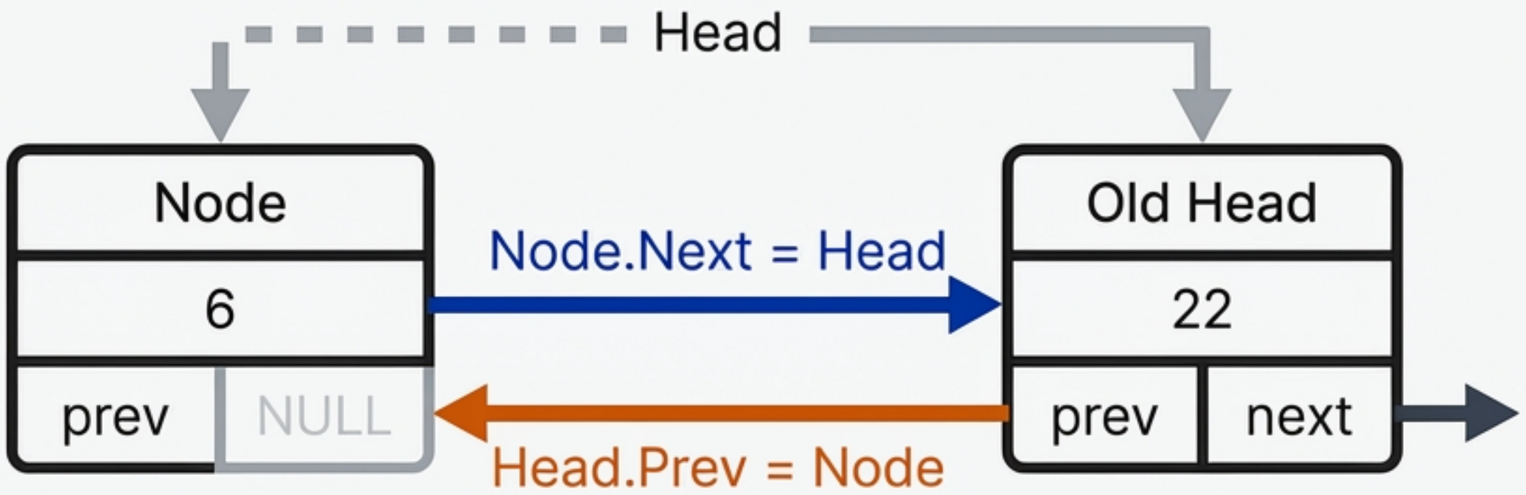
The Logic (Coding)

```
struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
};

// Initialize a new node
struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
newNode->data = 10;
newNode->prev = NULL; // Burnt Orange pointer concept
newNode->next = NULL; // International Klein Blue pointer concept

// Connecting nodes example (conceptual)
node1->next = node2;
node2->prev = node1;
node2->next = node3;
node3->prev = node2;
```

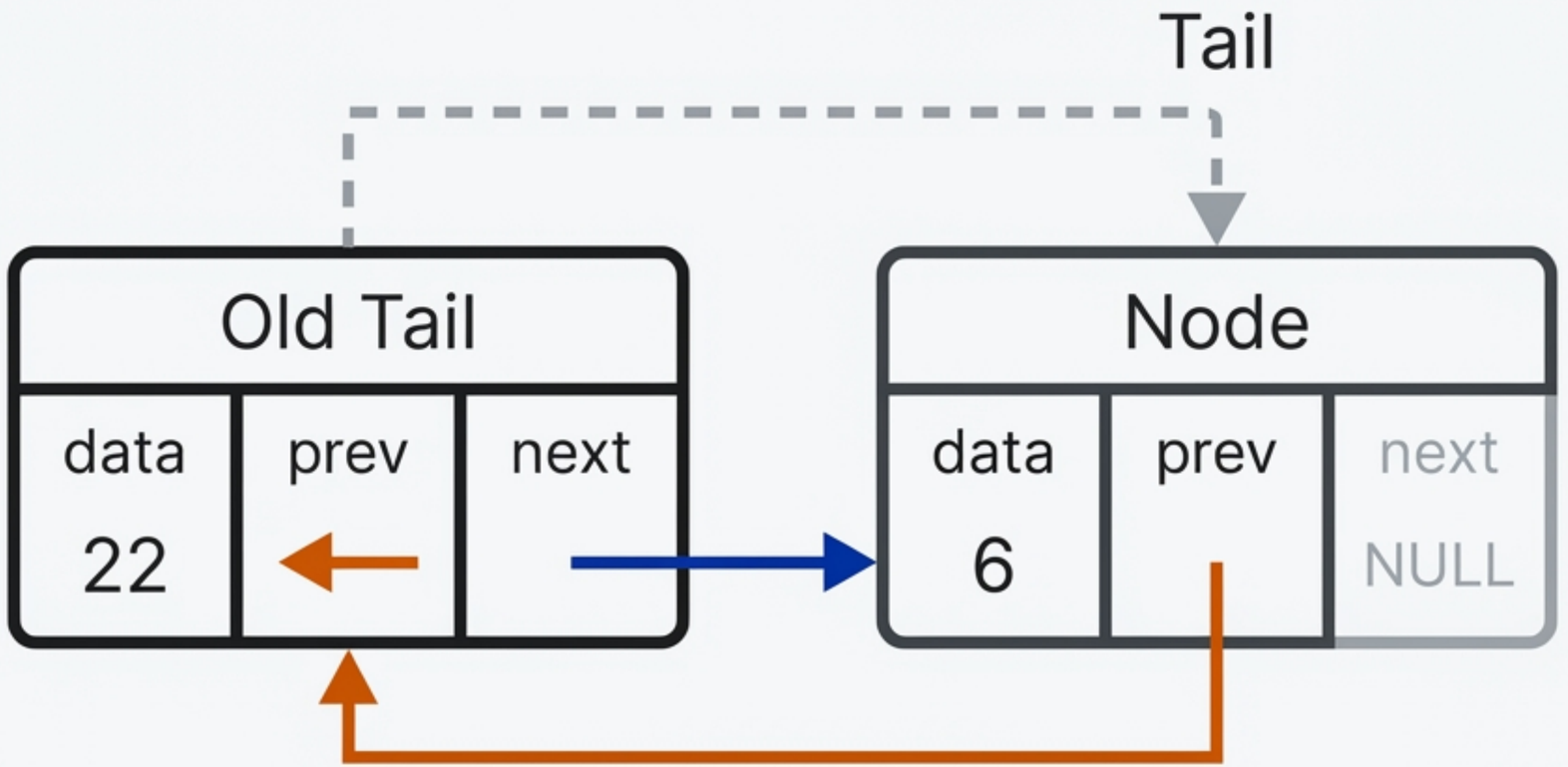
Operation: Insert at First




```
Node.Next = Head
Head.Prev = Node
Head = Node
Size++
```

Operation: Insert at Last

No traversal needed. We use the Tail reference for immediate O(1) access.

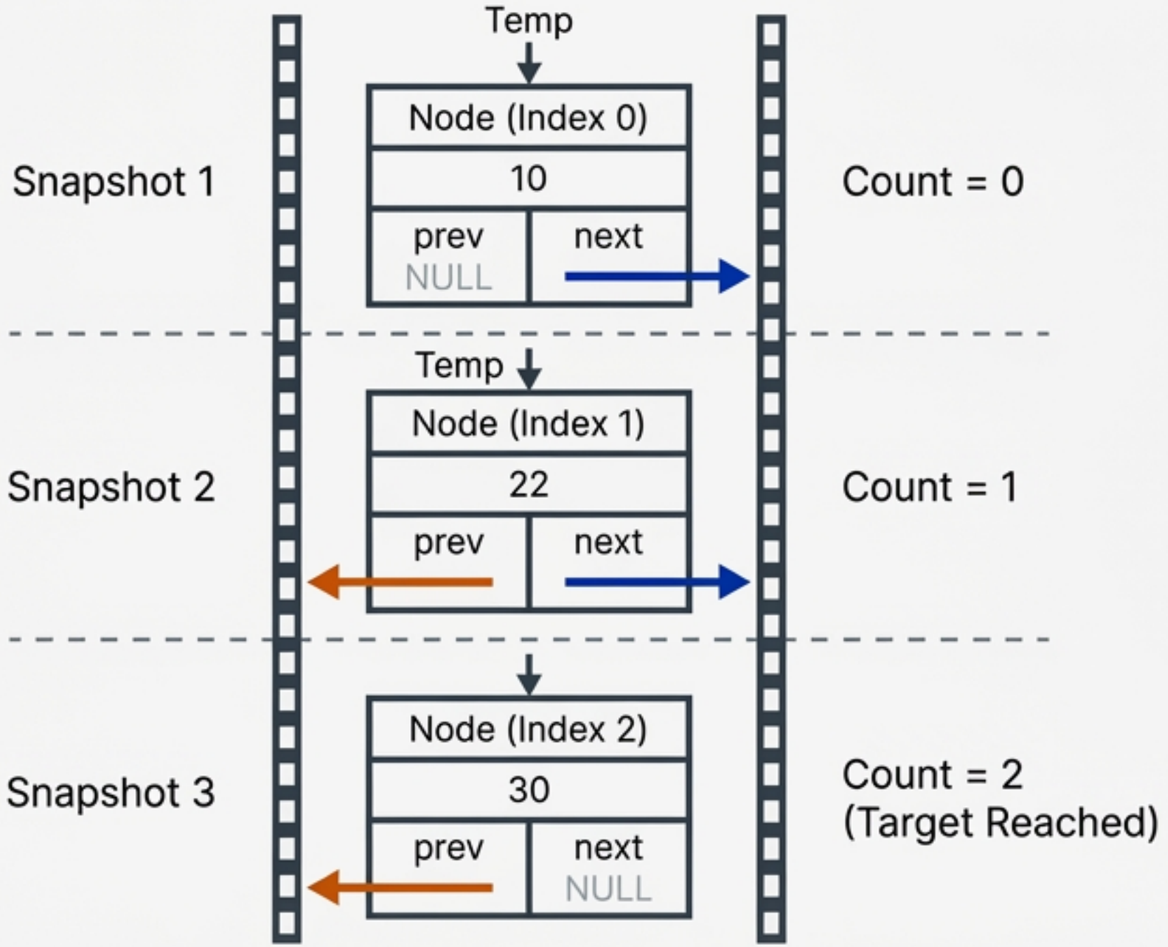



```
Tail.Next = Node
Node.Prev = Tail
Tail = Node
```

The Traversal Strategy

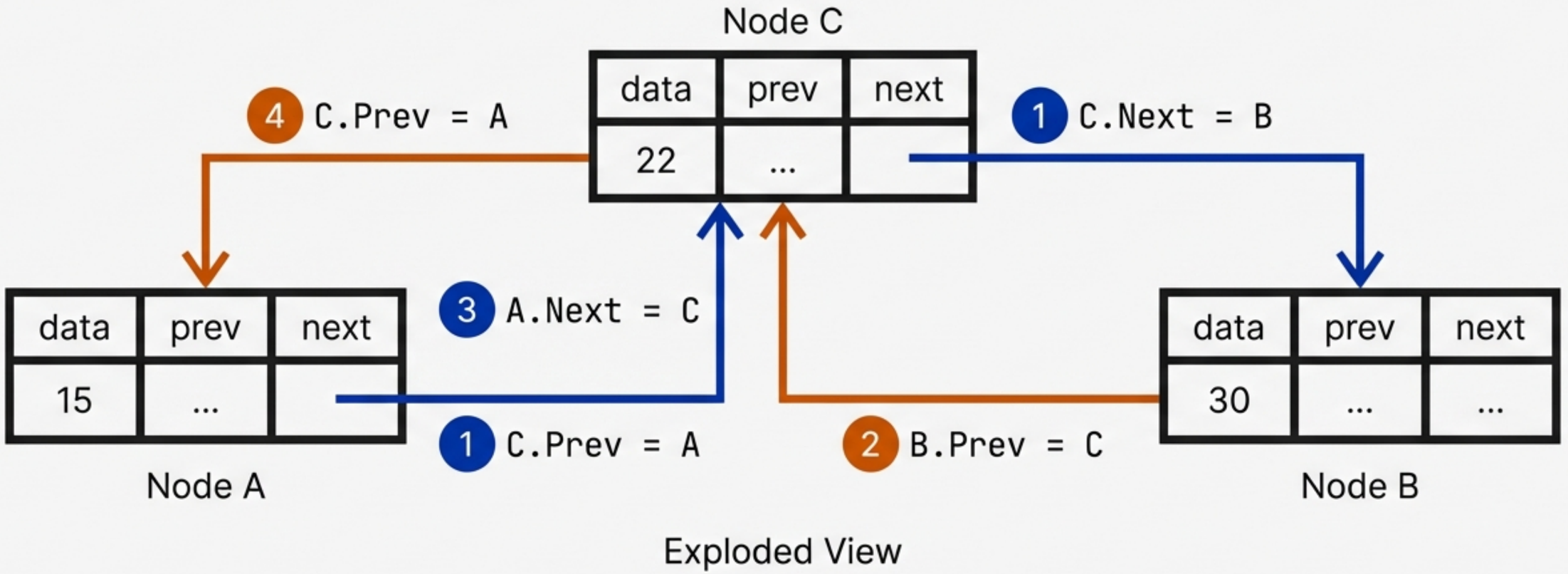
We use a `Temp` variable to protect the `Head`. We loop until `Count` equals `Index - 1`.

“Don’t think about the code. Just move the Temp variable.”



Operation: Insert at Index

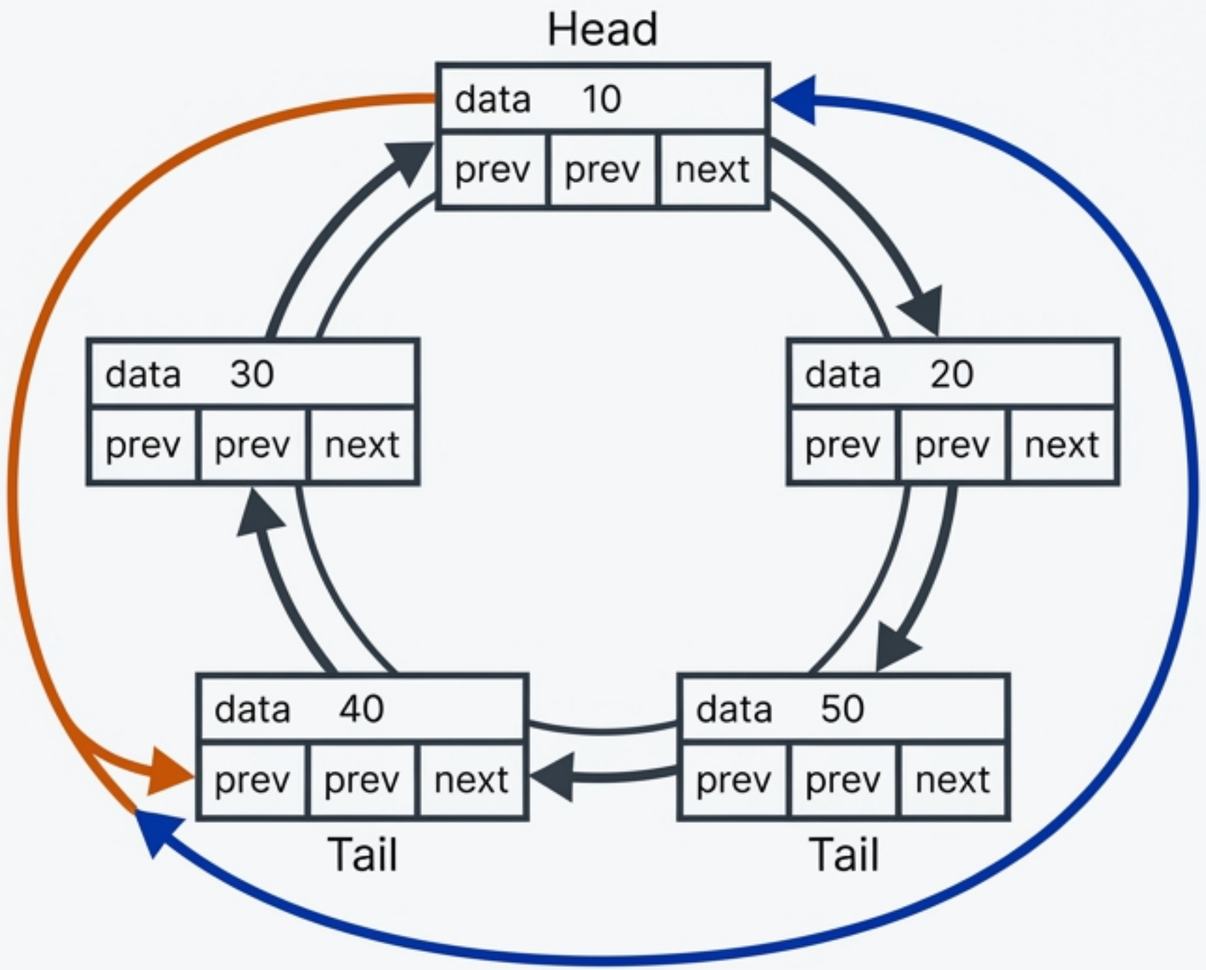
Precision matters. We must establish links to the new node *before* breaking the old links to avoid losing the rest of the list.



Circular Linked Lists: Breaking the Null

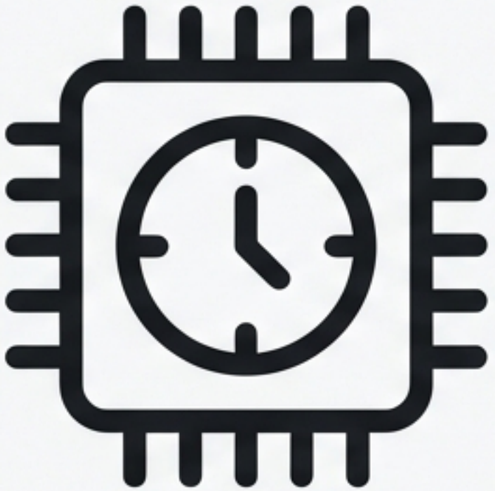
There is no Null pointer. The end connects to the start.

```
Tail.Next = Head
Head.Prev = Tail
```

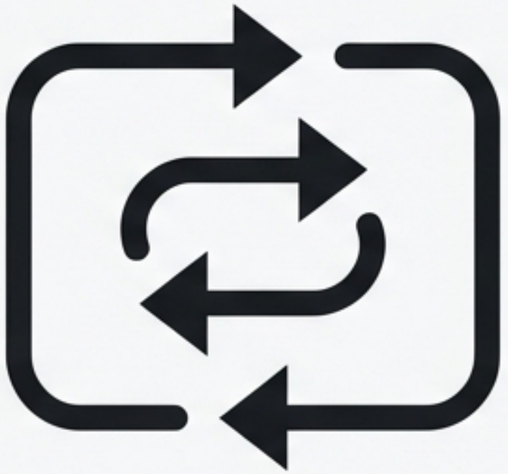


Why Go Circular?

Essential for applications requiring continuous looping or where start/end boundaries are undefined.



Round Robin Scheduling.

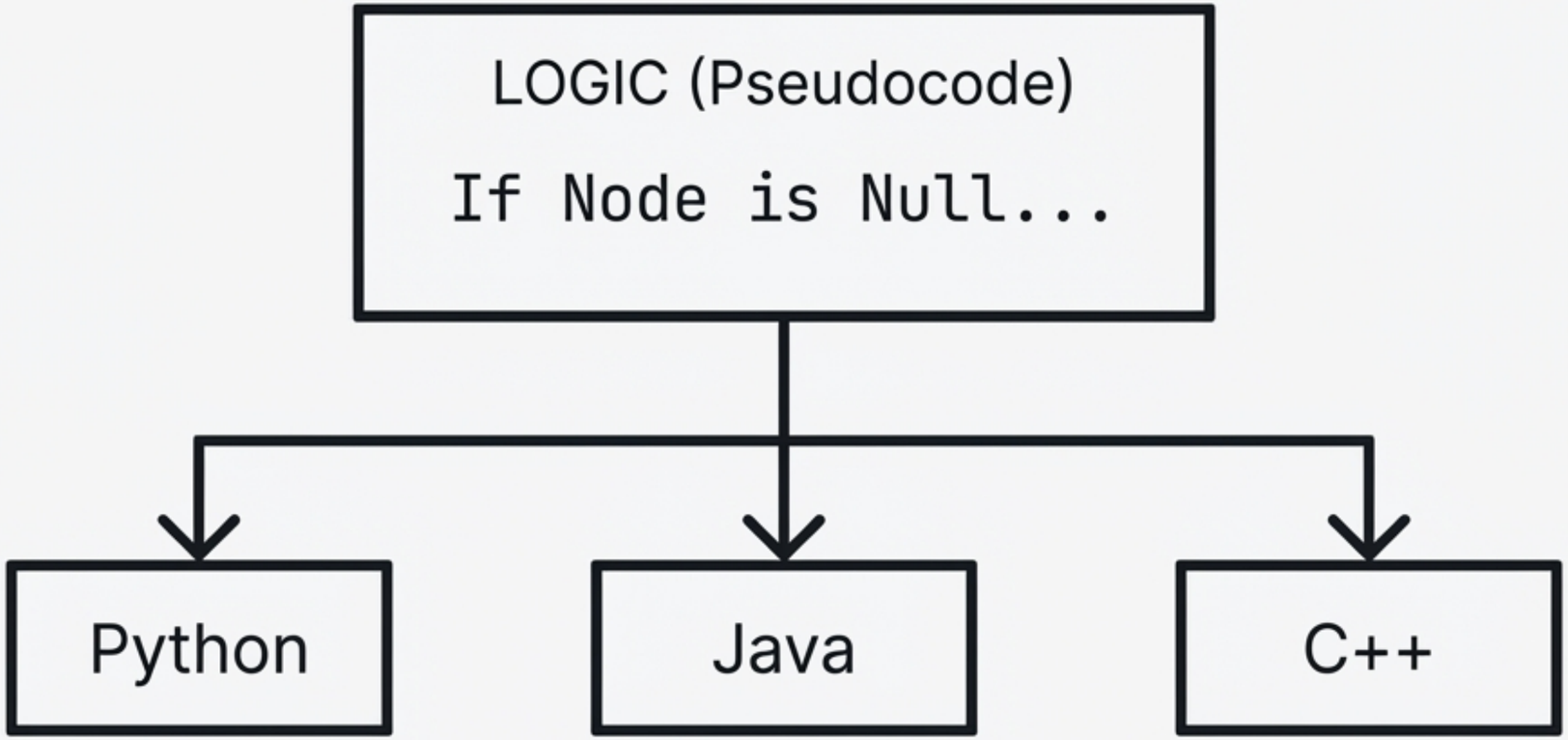


Infinite Playlists / Scroll.

Cycle Detection Algorithms.

Logic Over Syntax

“Language is just the mean of communication. If you can write the pseudocode, you can write it in any language.”

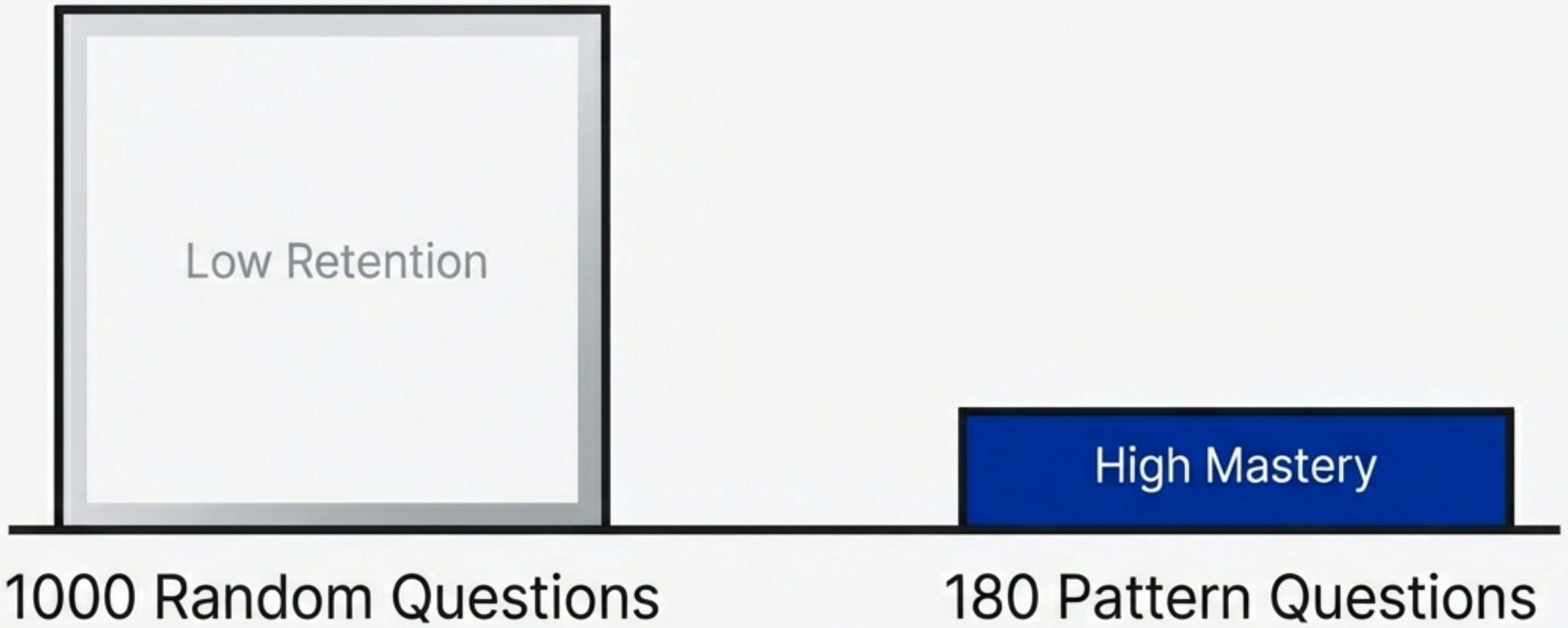


The Roadmap to Mastery

Stop solving random problems. Follow a structured path to build pattern recognition.

Quality > Quantity

Focus on patterns, not volume. Understanding the underlying structures is the key to cracking technical interviews.



Don’t Learn Alone.

“Resources are everywhere. Consistency and community are the variables.” Connect with peer groups to accelerate growth through discussion and code reviews.